

Chapter 6

Acceptance Testing User Stories

One reason for writing acceptance tests is to express many of the details that result from the conversations between customers and developers. Rather than writing lengthy lists of “The system shall...” style requirements statements, tests are used to fill in the details of a user story.

Testing is best viewed as a two-step process: First, notes about future tests are jotted on the back of story cards. This can be done any time someone thinks of a new test. Second, the test notes are turned into full-fledged tests that are used to demonstrate that the story has been correctly and fully coded.

As an example of test reminders you may write on the back of a card, the story “A company can pay for a job posting with a credit card” may have the following written on the back of its card:

- Test with Visa, MasterCard and American Express (pass).
- Test with Diner’s Club (fail).
- Test with good, bad and missing card ID numbers.
- Test with expired cards.
- Test with different purchase amounts (including one over the card’s limit).

These test notes capture assumptions made by the customer. Suppose the customer in the BigMoneyJobs example writes the story “A Job Seeker can view details about a specific job.” The customer and a developer discuss the story and identify a set of facts that will be displayed about a job—title, description, location, salary range, how to apply, and so on. However, the customer knows that not all companies will provide all of this information and she expects the site to handle missing data. For example, if no salary information is provided, the customer does not even want the “Salary range” label shown on the screen. This should be reflected as a test because the programmer may *assume* that the

job posting part of the system will require every job posting to include salary information.

Acceptance tests also provide basic criteria that can be used to determine if a story is fully implemented. Having criteria that tell us when something is done is the best way to avoid putting too much, or too little, time and effort into it. For example, when my wife bakes a cake, her acceptance test is to stick a toothpick into it; if the toothpick comes out clean, the cake is done. I acceptance test her cake by running a finger through the frosting and tasting.

Write Tests Before Coding

Acceptance tests provide a great deal of information that the programmers can use in advance of coding the story. For example, consider “Test with different purchase amounts (including one over the card’s limit).” If this test is written before a programmer starts coding, it will remind her to handle cases in which a purchase is declined because of insufficient credit. Without seeing that test, some programmers will forget to support this case.

Naturally, in order for programmers to benefit in this way, the acceptance tests for a story must be written before programming begins on that story. Tests are generally written at the following times:

- whenever the customer and developers talk about the story and want to capture explicit details
- as part of a dedicated effort at the start of an iteration but before programming begins
- whenever new tests are discovered during or after the programming of the story

Ideally, as the customer and developers discuss a story they reflect its details as tests. However, at the start of an iteration the customer should go through the stories and write any additional tests she can think of. A good way to do this is to look at each story and ask questions similar to the following:

- What else do the programmers need to know about this story?
- What am I assuming about how this story will be implemented?
- Are there circumstances when this story may behave differently?
- What can go wrong during the story?

Story Card 6.1 shows an example for a real project that was building software for a scanning system. The author of this story has clearly stated what she expects to happen (the newly-scanned pages go into a new document, even if a document is currently open in the software). In this case, the expectation was described as part of the story on the front of the card. It could just as easily have been stated as a test on the back of the card. The important thing is that the expectation is reflected somewhere on the card prior to the programmers starting on the story. If that is not done, it is likely the programmers could have coded different behavior, such as inserting the newly-scanned pages into the current document.

A user can scan pages and insert them into a new document. If a document is already open, then the app should prompt and close the current document.

■ Story Card 6.1 Conveying expectations to the programmers.

The Customer Specifies the Tests

Because the software is being written to fulfill a vision held by the customer, the acceptance tests need to be specified by the customer. The customer can work with a programmer or tester to actually create the tests, but minimally the customer needs to specify the tests that will be used to know when a story has been correctly developed. Additionally, a development team (especially one with experienced testers on it) will usually augment some of the stories with tests they think of.

Testing Is Part of the Process

I recently worked with a company where the tester got her understanding of the software from the programmers. The programmers would code a new feature, they'd explain it to the tester, and the tester would then validate that the program worked as described. Quite often the program would pass these tests but would then be plagued with errors once users started working with it. The problem, of course, was that the tester was testing that the programmer did

what she said she did. Without involvement from customers or users, no one was testing that the software did what they wanted it to do.

With user stories it is vital that testing be viewed as part of the development process, not something that happens “after coding is done.” Specifying tests is often a shared responsibility of a product manager and a tester. The product manager will bring her knowledge of the organizational goals driving the project; the tester will bring his suspicious mindset. At the start of an iteration they will get together and specify as many initial tests as they can think of. But it doesn’t stop there, and it doesn’t stop with them getting together once a week. As the details of a story are worked out, additional tests are specified.

How Many Tests Are Too Many?

The customer should continue to write tests as long as they add value and clarification to the story. It is probably not necessary to write a test to confirm that charges cannot be placed on an expired Visa card if you’ve already written such a test for expired MasterCards.

Also, keep in mind that a good programming team will have unit tests in place for many of the low-level cases. For example, the programming team should have unit tests that correctly identify February 30 and June 31 as invalid dates. The customer is not responsible for identifying every possible test. The customer should focus her efforts on writing tests that clarify the intent of the story to the developers.

The Framework for Integrated Test

Acceptance tests are meant to demonstrate that an application is acceptable to the customer who has been responsible for guiding the system’s development. This means that the customer should be the one to execute the acceptance tests. Minimally, acceptance tests should be executed at the end of each iteration. Because working code from one iteration may be broken by development in a subsequent iteration, it is important to execute acceptance tests from all prior iterations. This means that executing acceptance tests gets more time consuming with each passing iteration. If possible, the development team should look into automating some or all of the acceptance tests.

One excellent tool for automating acceptance tests is Ward Cunningham’s Framework for Integrated Test¹, or FIT for short. Using FIT, tests are written in

a familiar spreadsheet or tabular format. Bob and Micah Martin have led the development of FitNesse², an extension of FIT that makes test writing even easier.

FitNesse (which uses FIT) is rapidly becoming a very popular approach for writing acceptance tests on agile projects. Because tests are expressed in spreadsheet-like tables within web pages, the effort for customers to identify and write tests is greatly reduced. Table 6.1 is a sample of the type of table that can be processed by these tools. Each row represents one set of data. In this case the first data row identifies a Visa card that expires in May of 2005 and has the number 4123456789011. The final column indicates whether this card should pass a validity check in the application.³ In this case the card is expected to be considered valid by the application.

Table 6.1 *Testing for valid credit cards with a table that can be used by FIT and FitNesse.*

CardType	Expiration	Number	valid()
Visa	05/05	4123456789011	true
Visa	05/23	4123456789012349	false
MasterCard	12/04	5123456789012343	true
MasterCard	12/98	5123456789012345	false
MasterCard	12/05	42	false
American Express	4/05	341234567890127	true

In order to execute the tests in Table 6.1, a programmer on the team needs to write code to respond to simple FIT commands. She writes code that invokes code in the application being tested to determine the validity of the credit card. However, to write the acceptance tests, the customer only needs to create a simple table like this one that shows data values and expected results.

When the tests in Table 6.1 are run, the results are displayed by coloring the test columns (the last column in this example) either green (for a passed test) or red (for a failed test). FitNesse and FIT make it simple for either a customer or developer to run the acceptance tests.

1. The Framework for Integrated Test, FIT, is available at fit.c2.com.
2. FitNesse is available from fitnesse.org.
3. For information on credit card validity, see www.beachnet.com/~hstiles/card-type.html.

Types of Testing

There are many types of testing, and the customer and development team should work together to ensure that the appropriate types of testing are occurring. For most systems, story testing is largely functional testing, which ensures that the application functions as expected. There are, however, other types of testing to be considered. For example, you may want to consider any or all of the following:

- User interface testing, which ensures that all of the components of the user interface behave as expected
- Usability testing, which is done to ensure an application that can be easily used
- Performance testing, which is done to gauge how well the application will perform under various workloads
- Stress testing, in which the application is subjected to extreme values of users, transactions, or anything else that may put the application under stress

Testing for Bugs, Not Coverage

On an agile, story-driven project, testing is not the antagonistic activity that it becomes for many teams. There is no “I gotcha” mentality when a bug is found. There is no room for passing the blame when a bug makes it all the way to production. The highly-collaborative, we’re-all-in-this-together mentality of the team prevents this.

On an agile project, we test to find and eliminate bugs; we don’t necessarily pursue goals of 100% code coverage or testing of all boundary conditions. We use our intuition, our knowledge, and our past experience to guide our test effort.

Testing is done by whoever is best equipped to do the testing. A customer needs to specify the acceptance tests but she does that with assistance and information from the developers and dedicated testers. For example, consider the tests in Table 6.1. The only expired card that is tested is a MasterCard. If we were striving for complete coverage we would also need to test other card types. But when the customer speaks with the developers, she learns that (in this example) all cards are processed identically and testing an invalid card of one type is sufficient. Over time, through frequent communication and through seeing which types of tests fail, everyone on the project learns how best to focus the test effort.

Summary

- Acceptance tests are used to express details that result from conversations between a customer and a developer.
- Acceptance tests document assumptions about the story a customer has that may not have been discussed with a developer.
- Acceptance tests provide basic criteria that can be used to determine if a story is fully implemented.
- Acceptance tests should be written by the customer rather than by a developer.
- Acceptance tests are written before the programmer begins coding.
- Stop writing tests when additional tests will not help clarify the details or intent of the story.
- FIT and FitNesse are excellent tools for writing acceptance tests in a familiar table or spreadsheet format.

Developer Responsibilities

- You may be responsible for automating the execution of acceptance tests if your team chooses to do so.
- You are responsible for thinking about additional acceptance tests when you start development of a new story.
- You are responsible for unit testing your code so that acceptance tests do not need to be specified for all the minutiae of a story.

Customer Responsibilities

- You are responsible for writing the acceptance tests.
- You are responsible for executing the acceptance tests.

Questions

- 6.1 Who specifies the tests? Who helps?
- 6.2 Why specify tests before the stories are coded?